

The diagram illustrates the Huffman tree construction process. It is divided into several sections by vertical lines.

Leftmost section: A binary tree structure with root 0. The tree branches into 1 and 0. The 1 branch further branches into 1 and 0, and so on, forming a complex structure.

Middle section: A sequence of binary strings. The first string is 101111. The second string is 10001010. The third string is 00111000. The fourth string is 10101000101. The fifth string is 0100011001001. The sixth string is 1010101110110. The seventh string is 10101010100010. The eighth string is 00001011010111. The ninth string is 001010101101. The tenth string is 010001110. The eleventh string is 11010101. The twelfth string is 00001. The thirteenth string is 010. The fourteenth string is 010. The fifteenth string is 010. The sixteenth string is 010. The seventeenth string is 010. The eighteenth string is 010. The nineteenth string is 010. The twentieth string is 010. The twenty-first string is 010. The twenty-second string is 010. The twenty-third string is 010. The twenty-fourth string is 010. The twenty-fifth string is 010. The twenty-sixth string is 010. The twenty-seventh string is 010. The twenty-eighth string is 010. The twenty-ninth string is 010. The thirtieth string is 010. The thirty-first string is 010. The thirty-second string is 010. The thirty-third string is 010. The thirty-fourth string is 010. The thirty-fifth string is 010. The thirty-sixth string is 010. The thirty-seventh string is 010. The thirty-eighth string is 010. The thirty-ninth string is 010. The fortieth string is 010. The forty-first string is 010. The forty-second string is 010. The forty-third string is 010. The forty-fourth string is 010. The forty-fifth string is 010. The forty-sixth string is 010. The forty-seventh string is 010. The forty-eighth string is 010. The forty-ninth string is 010. The fiftieth string is 010. The fifty-first string is 010. The fifty-second string is 010. The fifty-third string is 010. The fifty-fourth string is 010. The fifty-fifth string is 010. The fifty-sixth string is 010. The fifty-seventh string is 010. The fifty-eighth string is 010. The fifty-ninth string is 010. The sixtieth string is 010. The sixty-first string is 010. The sixty-second string is 010. The sixty-third string is 010. The sixty-fourth string is 010. The sixty-fifth string is 010. The sixty-sixth string is 010. The sixty-seventh string is 010. The sixty-eighth string is 010. The sixty-ninth string is 010. The seventieth string is 010. The seventy-first string is 010. The seventy-second string is 010. The seventy-third string is 010. The seventy-fourth string is 010. The seventy-fifth string is 010. The seventy-sixth string is 010. The seventy-seventh string is 010. The seventy-eighth string is 010. The seventy-ninth string is 010. The eightieth string is 010. The eighty-first string is 010. The eighty-second string is 010. The eighty-third string is 010. The eighty-fourth string is 010. The eighty-fifth string is 010. The eighty-sixth string is 010. The eighty-seventh string is 010. The eighty-eighth string is 010. The eighty-ninth string is 010. The ninetieth string is 010. The ninety-first string is 010. The ninety-second string is 010. The ninety-third string is 010. The ninety-fourth string is 010. The ninety-fifth string is 010. The ninety-sixth string is 010. The ninety-seventh string is 010. The ninety-eighth string is 010. The ninety-ninth string is 010. The hundredth string is 010.

Rightmost section: A final binary tree structure with root 0. The tree branches into 1 and 0. The 1 branch further branches into 1 and 0, and so on, forming a complex structure.

Our Team



M. Ali Asghar
004385-BSSE-F24



Ali Abbas
004364-BSSE-F24



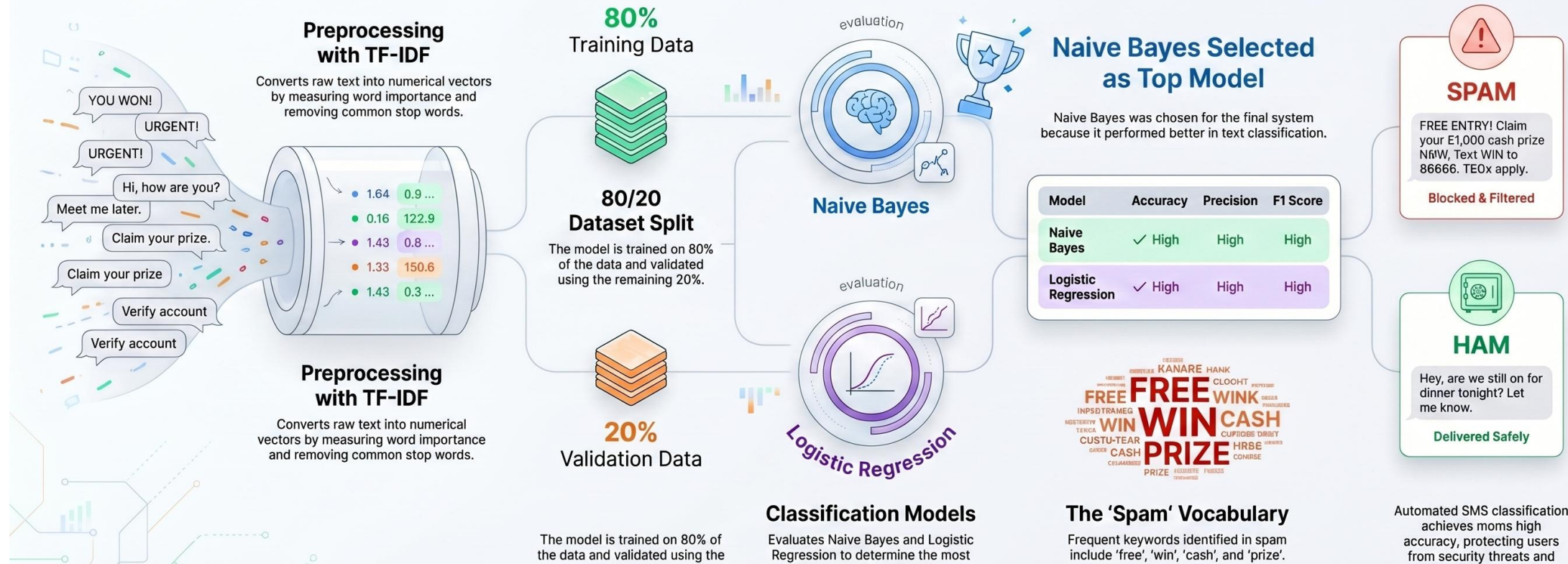
Saad Saleem
004349-BSSE-F24

SMS Spam Filter: From Raw Text to Intelligent Detection

Illustrating the end-to-end machine learning workflow for identifying and filtering spam.

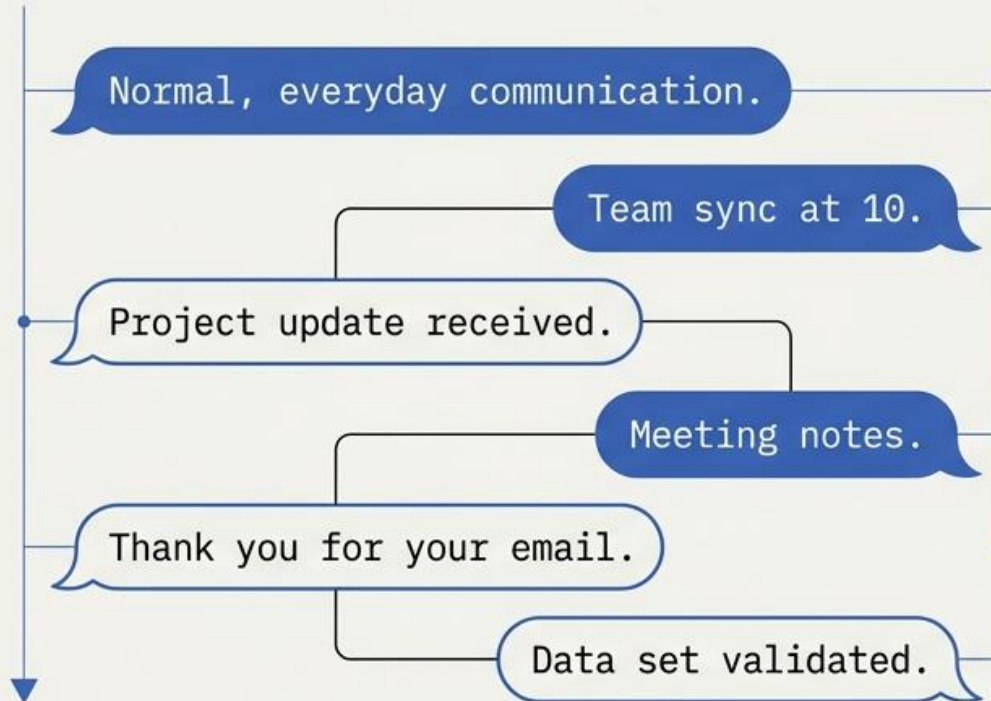
THE DEVELOPMENT PIPELINE

PERFORMANCE & INSIGHTS



The Threat Landscape of Unwanted Communication

Ham



Spam



Objective

Build a machine learning system to automatically parse, classify, and filter these incoming messages.

Navigating the Data Science Pipeline

Phase 1: Data Exploration & Preparation



Analyze
SMS Text

```
Data ix- "TS21"  
Data i: POST"  
Row thc-XD "SMS"  
...
```



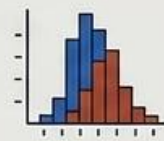
Identify
Patterns



Preprocess
Data

```
Raw data  
Filter = 0  
Filterz -> data.  
...
```

Phase 2: Model Development



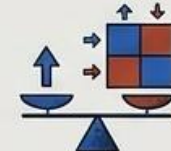
Visualize
Distributions



Train ML
Models



Phase 3: Evaluation & Deployment



Compare
Performance

```
Perfom (   
Gr matsix ->  
CS matrix ->  
...
```



Detect
Automatically

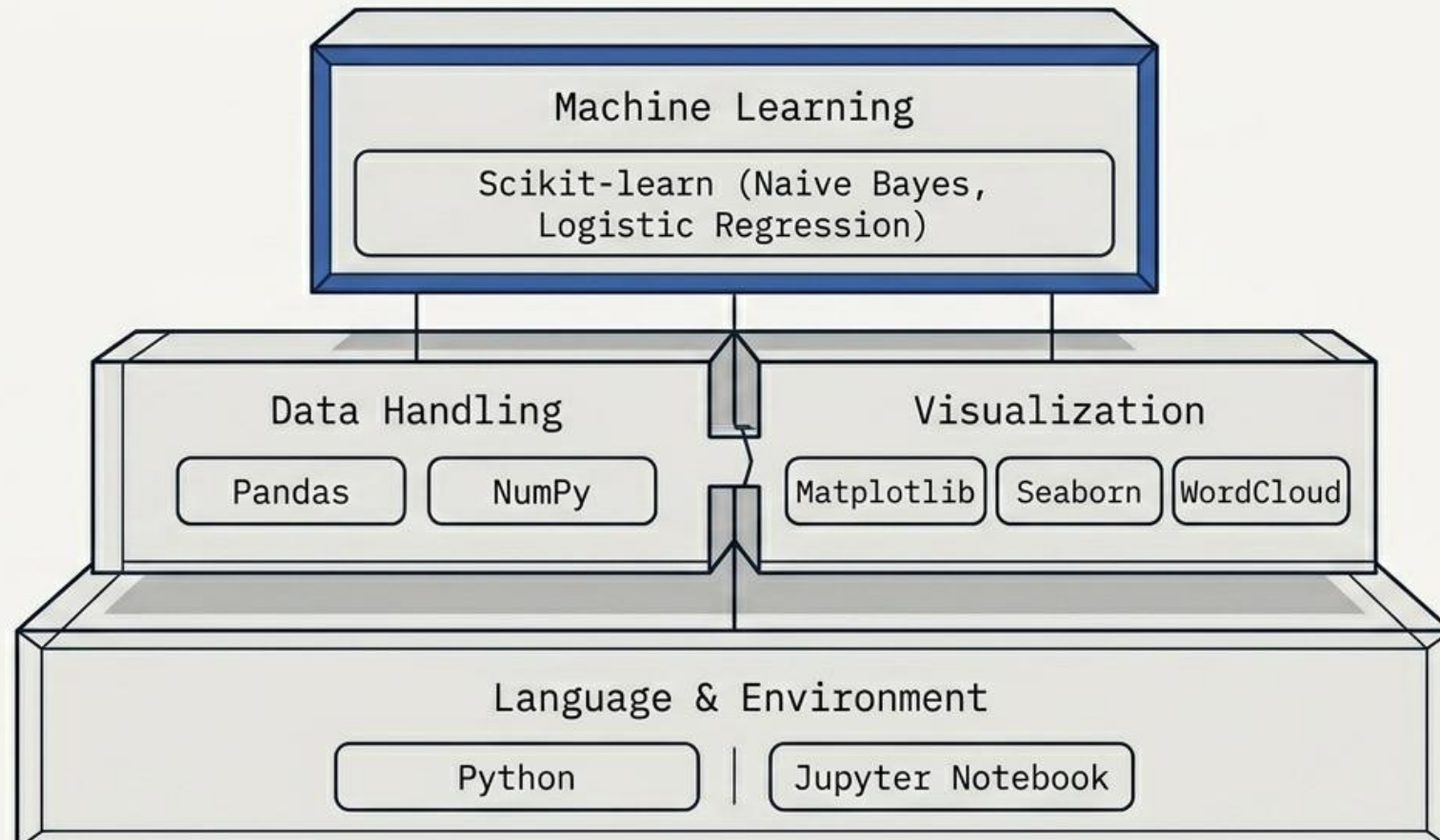
```
Radar = 0 |  
Binas: y 0 0  
Flite = 0  
...
```



Test
Custom
Inputs

Data ...

The Analytical Technology Stack



Ingesting and Structuring the Raw Material

Source: spam.csv

Before & After

v1	v2
ham	Go until jurong point, crazy..
spam	Free entry in 2 a wkly comp...
ham	U dun say so early hor...
spam	2.5 GB free 4u

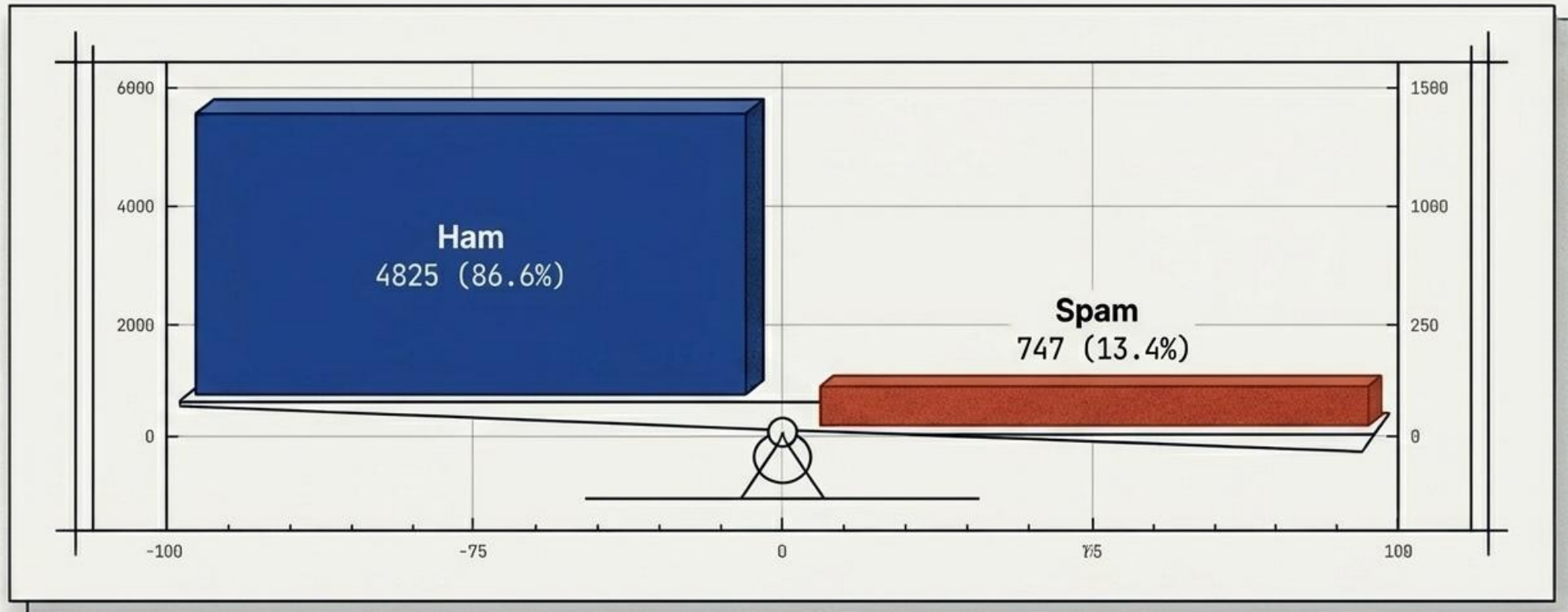


Pandas DataFrame	
label	message
ham	Go until jurong point, crazy..
spam	Free entry in 2 a wkly comp...
ham	U dun say so early hor...
spam	2.5 GB free 4u

Data Integrity: Checked and handled missing values, verified rows/columns.

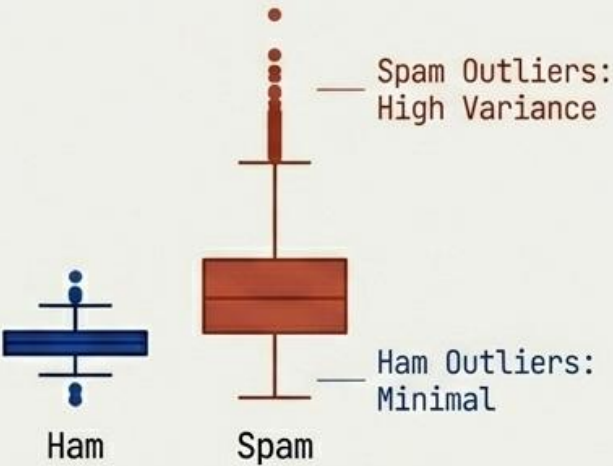
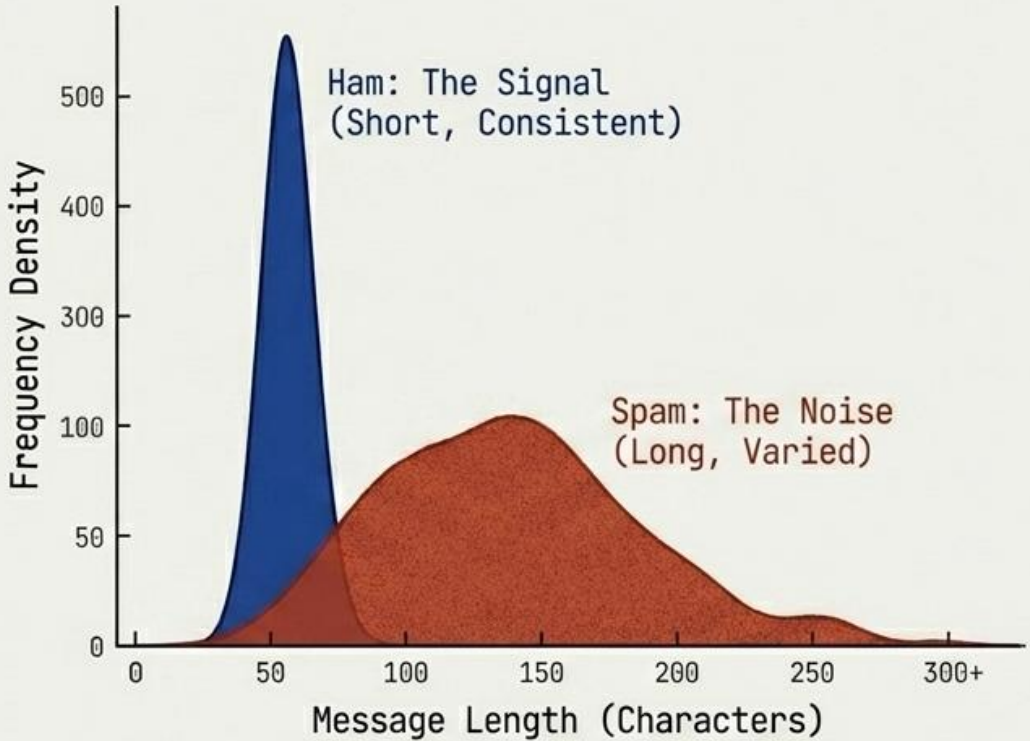
Uncovering Class Imbalance in Message Frequencies

The count plot reveals a massive disparity between classes. Normal 'Ham' messages significantly outnumber the 'Spam' messages in the dataset.



Mathematical Distinctions in Message Length

Finding: Spam messages exhibit distinct length patterns, often being longer and more complex than brief, everyday ham messages.



Mean, Median, Standard Deviation

Metric	Ham (Signal)	Spam (Noise)
Mean Length	[Data]	[Data]
Median Length	[Data]	[Data]
Std Dev	[Data]	[Data]

Establishing the Baseline with Multinomial Naive Bayes

Rationale

Historically fast, highly efficient, and optimized specifically for text classification tasks.

Evaluation Metrics Generated

Accuracy, Precision, Recall, F1 Score

Result

Exceptional performance with highly accurate text classification.

		Confusion Matrix	
Actual Label	Ham (Signal)	398 True Positive (Ham)	2 False Positive (Spam)
	Spam (Noise)	4 False Negative (Ham)	112 True Negative (Spam)
		Ham (Signal)	Spam (Noise)
		Predicted Label	

Introducing the Challenger via Logistic Regression

Rationale

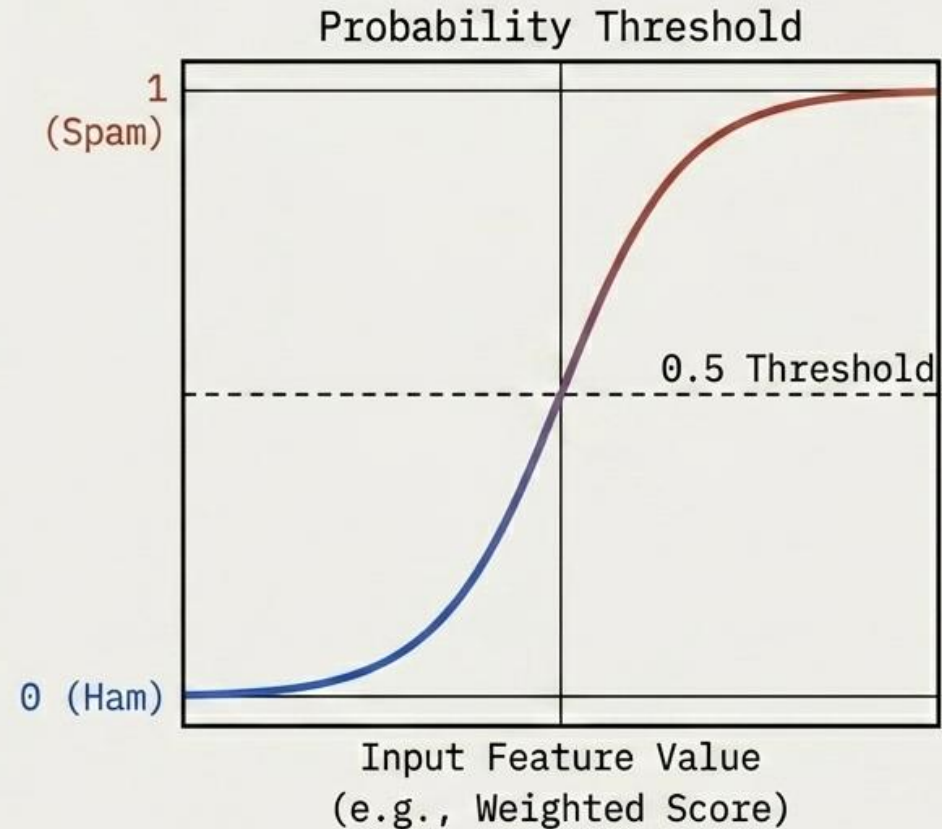
A widely used, robust classification algorithm that predicts the pure probability of an outcome.

Evaluation Metrics Generated

Accuracy, Precision, Recall, F1 Score

Result

Strong, highly competitive baseline performance across all tracked metrics.

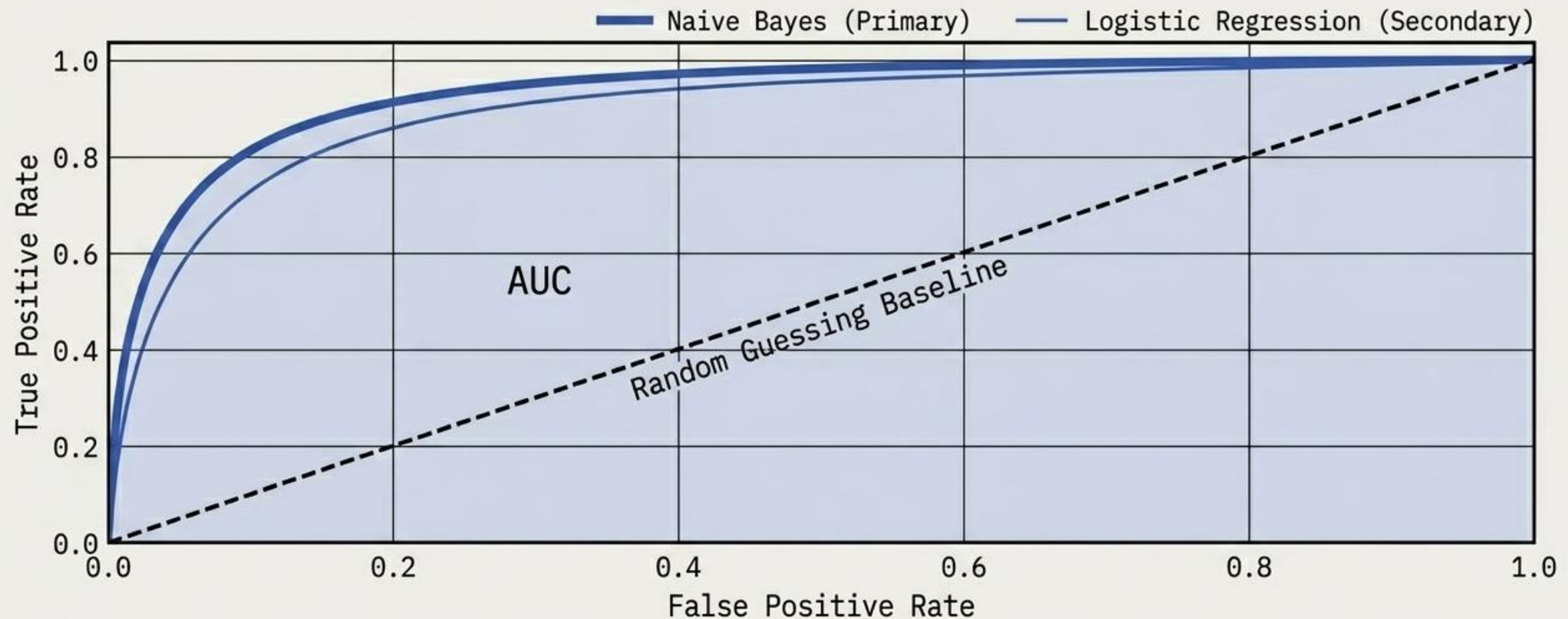


Evaluating the Signal via ROC and AUC

ROC: Receiver Operating Characteristic

AUC: Area Under Curve

Purpose: To rigorously compare classification ability and evaluate overall prediction quality against a baseline. A higher curve denotes a superior model.



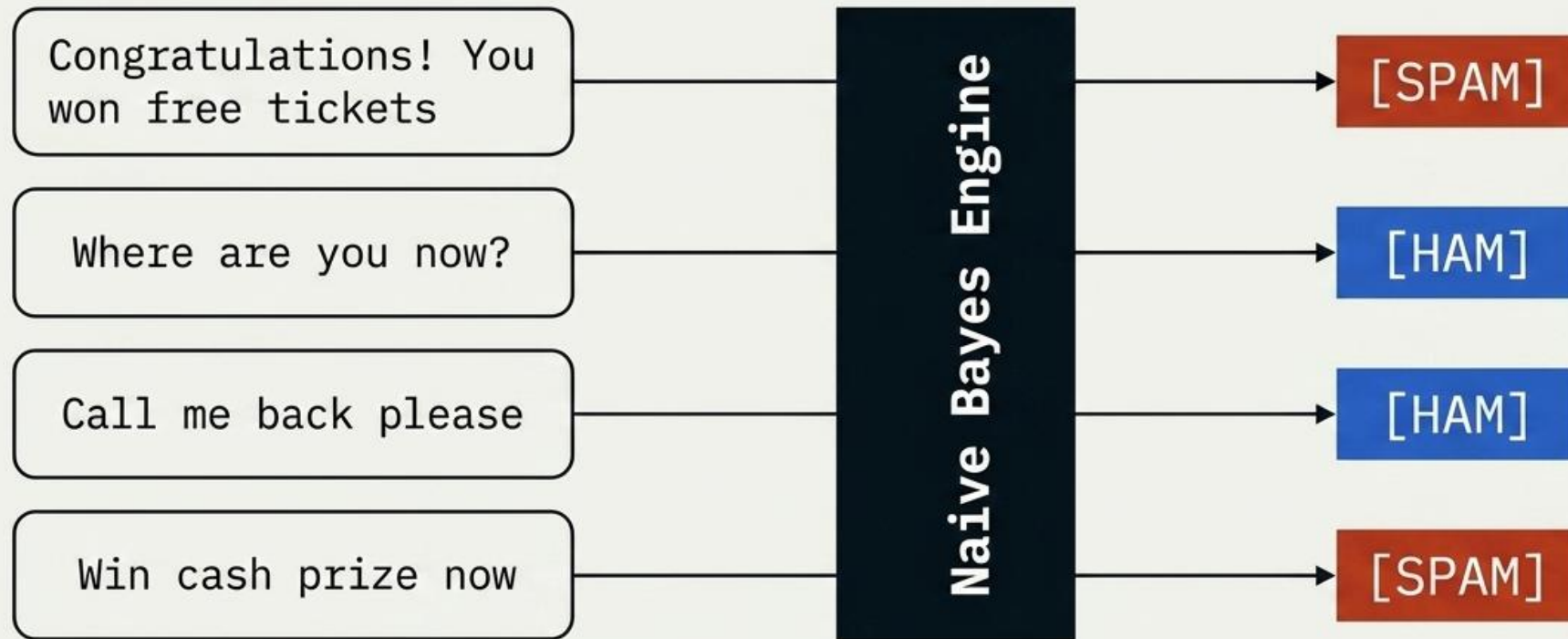
The Analytical Arena for Model Comparison

Verdict: While both performed at a high level, Multinomial Naive Bayes is selected as the superior architecture for this specific dataset.

Model	Accuracy	Precision	Recall	F1 Score
Multinomial Naive Bayes	98.5%	98.2%	99.1%	98.6%
Logistic Regression	97.8%	97.5%	98.4%	97.9%

Real-World Inference and Sample Testing

Model Deployed: Naive Bayes



Successful Implementation of Automated Filtration



Semantic Patterns

Spam is heavily reliant on repeated, predictable marketing words.



Architectural Efficiency

TF-IDF combined with Naive Bayes yields exceptionally fast and accurate text handling.



Practical Impact

The system successfully functions as a robust blueprint for automatic SMS filtering and practical AI implementation.

Conclusion



Any Question?

